

Panduan Deploy SEMS

Smart Energy Management System

Panduan lengkap pemasangan untuk firmware ESP32,
backend Google Apps Script, dan frontend Next.js

VERSI KOMPONEN

Firmware v7.0.0 | Backend v1.5.0 | Frontend v0.5.0

PT. Jaya Mandiri Smart Energy

Jambi Solar Panel Division

Daftar Isi

1. Pendahuluan	3
1.1 Tentang SEMS	3
1.2 Arsitektur Sistem	3
1.3 Versi Komponen	4
2. Persyaratan Sistem	4
2.1 Hardware	4
2.2 Software	5
2.3 Akun yang Diperlukan	6
3. Deploy Backend - Google Apps Script	6
3.1 Buat Google Spreadsheet Baru	6
3.2 Buat Proyek Google Apps Script	7
3.3 Copy File Backend (.gs)	7
3.4 Ubah SPREADSHEET_ID di Config.gs	8
3.5 Jalankan setupSheets_()	8
3.6 Deploy sebagai Web App	8
3.7 Catat URL Deployment	9
4. Deploy Frontend - Next.js	9
4.1 Opsi A: Deploy ke Vercel (Paling Mudah)	9

4.1.1 Install Dependencies	9
4.1.2 Buat File .env.local	10
4.1.3 Build dan Deploy	10
4.2 Opsi B: Deploy ke VPS dengan Caddy	10
4.2.1 Install dan Build di VPS	10
4.2.2 Konfigurasi Caddy	11
4.2.3 Jalankan Service	11
5. Deploy Firmware - ESP32	12
5.1 Install PlatformIO	12
5.2 Buka Project Firmware	12
5.3 Konfigurasi Credentials	12
5.4 Upload ke ESP32	13
5.5 Verifikasi Firmware	13
6. Konfigurasi dan Pengujian	14
6.1 Buat User Admin Pertama	14
6.2 Konfigurasi Device Token	14
6.3 Verifikasi Koneksi Firmware ke Backend	15
6.4 Test Frontend Dashboard	15
7. Troubleshooting	15
8. Keamanan - Checklist Wajib	16

1. Pendahuluan

1.1 Tentang SEMS

Smart Energy Management System (SEMS) adalah solusi IoT terintegrasi yang dirancang untuk monitoring, manajemen, dan optimasi energi surya secara realtime. Sistem ini mengelola seluruh rantai energi dari panel surya, baterai LiFePO4 8S (25.6V nominal), hingga distribusi daya ke berbagai perangkat rumah tangga melalui relay dan MOSFET PWM. SEMS dikembangkan oleh PT. Jaya Mandiri Smart Energy di bawah brand Jambi Solar Panel untuk memenuhi kebutuhan manajemen energi yang cerdas, efisien, dan andal.

Dengan SEMS, pengguna dapat memantau tegangan dan arus baterai, status masing-masing sel (cell) LiFePO4, arus inverter, suhu dan kelembaban ruangan, hingga deteksi gerakan melalui sensor PIR. Semua data dikirimkan oleh firmware ESP32 ke backend Google Apps Script secara periodik setiap 15 detik, kemudian ditampilkan secara realtime di dashboard Next.js. Fitur otomasi, penjadwalan, alarm, load shedding, dan notifikasi email turut melengkapi kemampuan sistem ini dalam menjaga keamanan dan efisiensi penggunaan energi.

Panduan ini ditulis untuk pengguna awam (non-programmer) dengan bahasa yang sederhana dan langkah-langkah yang detail. Anda tidak perlu memiliki keahlian pemrograman untuk mengikuti panduan ini, namun pemahaman dasar tentang penggunaan komputer dan browser web akan sangat membantu.

1.2 Arsitektur Sistem

SEMS terdiri dari tiga komponen utama yang saling terintegrasi. Arsitektur ini dirancang agar setiap komponen dapat dikelola secara independen namun tetap terhubung melalui API berbasis HTTP/HTTPS. Berikut adalah deskripsi arsitektur ketiga komponen tersebut:

```
+-----+
| ARSITEKTUR SEMS |
+-----+
| |
| [Sensor & Relay] |
| INA219, ADS1115, SHT31, DS3231, ACS712, PIR |
| | |
| v |
| +-----+ |
| | ESP32 Firmware v7.0.0 | <-- Hardware Mikrokontroler |
| | (platformio.ini) | Membaca sensor, mengirim |
| | +-----+ data ke backend via HTTP |
| | |
| | HTTP POST (telemetry setiap 15 detik) |
| v |
| +-----+ |
| | Google Apps Script v1.5 | <-- Backend berbasis Cloud |
```

```

| | (13 file .gs) | Menyimpan data ke |
| +-----+ Google Spreadsheet, |
| | menangani auth, alarm, |
| | HTTP GET/POST konfigurasi, dan API |
| v |
| +-----+ |
| | Next.js Frontend v0.5.0 | <-- Dashboard Web |
| | (Vercel / Caddy + VPS) | Menampilkan data, |
| +-----+ kontrol relay, settings |
| |
+-----+

```

1.3 Versi Komponen

Setiap komponen SEMS memiliki versi tersendiri. Pastikan Anda menggunakan versi yang sesuai dengan panduan ini untuk menghindari masalah kompatibilitas. Berikut adalah detail versi masing-masing komponen beserta peran utamanya dalam sistem:

Komponen	Versi	Deskripsi
ESP32 Firmware	v7.0.0	Membaca sensor INA219, ADS1115, SHT31, DS3231, ACS712, PIR. Mengirim data telemetry setiap 15 detik. Mengelola relay via 74HC595 shift register. Mendukung OTA update, load shedding, dan automation engine.
Google Apps Script	v1.5.0	Backend berbasis cloud. Menyimpan semua data di Google Spreadsheet. Menyediakan REST API untuk frontend dan firmware. Menangani autentikasi, otorisasi role (admin/technician/viewer), alarm, notifikasi email, dan pembersihan data otomatis.
Next.js Frontend	v0.5.0	Dashboard web menggunakan Next.js 16, React 19, Tailwind CSS, dan shadcn/ui. Menampilkan data realtime, grafik telemetry, kontrol relay, penjadwalan, manajemen user, dan halaman konfigurasi sistem.

2. Persyaratan Sistem

Sebelum memulai proses deployment, pastikan semua persyaratan berikut telah terpenuhi. Persyaratan ini mencakup hardware, software, dan akun yang diperlukan untuk menjalankan seluruh komponen SEMS. Periksa setiap item dengan teliti untuk menghindari masalah di tengah proses instalasi.

2.1 Hardware

Hardware yang dibutuhkan adalah mikrokontroler ESP32 WROOM (devkit) beserta sensor-sensor pendukung dan modul relay. ESP32 berfungsi sebagai otak sistem yang membaca semua sensor dan mengirimkan data ke backend cloud. Pastikan Anda memiliki kabel USB micro-USB atau USB-C yang berfungsi baik untuk menghubungkan ESP32 ke komputer saat proses upload firmware.

Komponen	Jumlah	Keterangan
ESP32 WROOM DevKit	1 unit	Mikrokontroler utama, board ESP32 devkit versi apapun
Kabel USB	1 buah	Micro-USB atau USB-C, untuk upload firmware dan power
Sensor INA219	1 unit	I2C address 0x40, mengukur tegangan dan arus baterai
Sensor ADS1115	2 unit	I2C address 0x48 dan 0x49, membaca 8 sel baterai
Sensor SHT31	1 unit	I2C address 0x44, suhu dan kelembaban ruangan
RTC DS3231	1 unit	I2C address 0x68, jam realtime dengan backup baterai
Sensor ACS712-30A	1 unit	GPIO 34, mengukur arus AC via efek Hall
Sensor PIR	4 unit	GPIO 32, 33, 25, 26, deteksi gerakan di 4 zona
74HC595 Shift Register	4 unit	32 bit, mengendalikan 13 relay + 4 MOSFET PWM
Relay Board	13 channel	Active-low, 220V AC, untuk kontrol perangkat listrik
Baterai LiFePO4 8S	1 pack	25.6V nominal, 29.2V full, 100Ah kapasitas

2.2 Software

Software yang dibutuhkan tergantung pada komponen mana yang akan Anda deploy. Untuk firmware, Anda memerlukan IDE PlatformIO. Untuk frontend, Anda memerlukan runtime Node.js atau Bun. Berikut daftar lengkap software beserta versi minimum yang direkomendasikan:

Software	Versi Min.	Untuk	Catatan
VS Code	Latest	Firmware	IDE utama, download dari code.visualstudio.com
PlatformIO Extension	Latest	Firmware	Install di VS Code melalui Extensions

Google Chrome / Firefox	Latest	Backend + Frontend	Browser modern untuk akses Spreadsheet
Node.js	18.x+	Frontend	Alternatif: Bun runtime
Bun	1.x+	Frontend	Runtime JS alternatif, lebih cepat dari Node.js
Git	2.x+	Frontend	Untuk push ke GitHub (jika deploy via Vercel)
Caddy Server	2.x+	Frontend (VPS)	Web server dengan HTTPS otomatis

2.3 Akun yang Diperlukan

Anda memerlukan beberapa akun online untuk menjalankan SEMS secara penuh. Akun Google wajib dimiliki karena backend berjalan di atas Google Apps Script dan menyimpan data di Google Spreadsheet. Akun Vercel bersifat opsional jika Anda ingin deploy frontend ke cloud secara gratis; sebagai alternatif, Anda bisa menggunakan VPS dengan Caddy. Berikut rincian lengkap akun yang diperlukan beserta fungsinya dalam sistem:

- **Google Account (wajib)** - Digunakan untuk Google Spreadsheet sebagai database, Google Apps Script sebagai backend API, dan Gmail untuk notifikasi email. Buat akun gratis di accounts.google.com jika belum memiliki.
- **Akun Vercel (opsional)** - Digunakan untuk hosting frontend Next.js secara gratis. Buat akun di vercel.com, hubungkan dengan GitHub, dan deploy otomatis setiap push. Jika tidak ingin menggunakan cloud, gunakan VPS dengan Caddy sebagai alternatif.

3. Deploy Backend - Google Apps Script

Backend SEMS menggunakan Google Apps Script (GAS) sebagai serverless API yang menyimpan semua data di Google Spreadsheet. Tidak diperlukan server fisik atau VPS untuk menjalankan backend ini. Proses deploy relatif sederhana dan hanya memerlukan browser web. Pastikan Anda sudah login ke Google Account sebelum memulai langkah-langkah di bawah ini.

3.1 Buat Google Spreadsheet Baru

Langkah pertama adalah membuat Google Spreadsheet baru yang akan berfungsi sebagai database untuk seluruh data SEMS. Buka browser Chrome atau Firefox, kunjungi sheets.google.com, dan login dengan akun Google Anda. Klik tombol "Blank" untuk membuat spreadsheet kosong baru. Beri nama spreadsheet dengan nama yang mudah diingat, misalnya "SEMS Database". Setelah spreadsheet terbuka, salin URL dari address bar browser Anda. Bagian URL yang Anda perlukan adalah

SPREADSHEET_ID, yaitu string karakter panjang yang terletak antara /d/ dan /edit pada URL.

URL: <https://docs.google.com/spreadsheets/d/1aBcDeFgHiJkLmNoPqRsTuVwXyZ/edit>

~~~~~

Ini adalah SPREADSHEET\_ID Anda

### 3.2 Buat Proyek Google Apps Script

Setelah spreadsheet dibuat, Anda perlu membuat proyek Google Apps Script yang terhubung dengan spreadsheet tersebut. Di spreadsheet yang sudah dibuka, klik menu "Extensions" di toolbar atas, lalu pilih "Apps Script". Sebuah tab baru akan terbuka menampilkan editor Apps Script. Di sinilah Anda akan menyalin semua file backend. Biarkan nama proyek secara default atau ubah sesuai keinginan Anda, misalnya menjadi "SEMS Backend v1.5.0".

### 3.3 Copy File Backend (.gs)

Anda perlu menyalin 13 file Google Apps Script (.gs) ke dalam proyek yang baru dibuat. File-file ini membentuk seluruh logic backend SEMS, mulai dari konfigurasi, autentikasi, hingga API untuk telemetry dan alarm. Klik tombol "+" di samping kiri editor untuk menambahkan file baru, beri nama sesuai daftar di bawah, lalu salin isi setiap file dari folder backend\_v7/ atau backend\_final/ di project Anda.

| N<br>o. | Nama File            | Fungsi Utama                                                        |
|---------|----------------------|---------------------------------------------------------------------|
| 1       | Code.gs              | Entry point, routing request, function doGet() dan doPost()         |
| 2       | Config.gs            | Konfigurasi: SPREADSHEET_ID, sheet names, headers, thresholds       |
| 3       | Auth.gs              | Autentikasi user, hash password, manajemen session via CacheService |
| 4       | DeviceAPI.gs         | CRUD devices: daftar, tambah, edit, hapus perangkat relay/MOSFET    |
| 5       | AlarmAPI.gs          | Manajemen alarm: list, acknowledge, trigger alarm otomatis          |
| 6       | TelemetryAPI.gs      | Terima data dari ESP32, simpan ke sheet, deduplikasi                |
| 7       | UserAPI.gs           | CRUD users: buat, edit, hapus user, management role                 |
| 8       | RuleAPI.gs           | Automation rules: CRUD aturan otomasi berdasarkan kondisi sensor    |
| 9       | ScheduleAPI.gs       | Penjadwalan: CRUD jadwal ON/OFF perangkat berdasarkan waktu         |
| 10      | ConfigAPI.gs         | Konfigurasi sistem: baca/tulis setting di sheet configurations      |
| 11      | EmailNotification.gs | Kirim notifikasi email saat alarm terjadi                           |



|    |                |                                                               |
|----|----------------|---------------------------------------------------------------|
| 12 | SetupSheets.gs | Inisialisasi otomatis: buat semua sheet, header, data default |
| 13 | DataCleanup.gs | Pembersihan data lama: hapus telemetry dan alarm expired      |

### 3.4 Ubah SPREADSHEET\_ID di Config.gs

Buka file Config.gs di editor Apps Script. Pada baris paling atas, Anda akan menemukan variabel SPREADSHEET\_ID yang masih berisi placeholder. Ganti nilai placeholder tersebut dengan SPREADSHEET\_ID yang Anda salin dari URL spreadsheet di langkah 3.1. Pastikan tidak ada spasi tambahan dan tanda kutip tetap ada. Perhatikan contoh perubahan berikut ini:

```
// SEBELUM (placeholder):
var SPREADSHEET_ID = 'REPLACE_WITH_YOUR_SPREADSHEET_ID';

// SESUDAH (ganti dengan ID Anda):
var SPREADSHEET_ID = '1aBcDeFgHiJkLmNoPqRsTuVwXyZ';
```

### 3.5 Jalankan setupSheets\_()

Sebelum backend bisa beroperasi, Anda perlu menginisialisasi semua sheet yang diperlukan. Buka file SetupSheets.gs, lalu klik tombol "Run" di toolbar atas editor. Anda akan diminta untuk memberikan izin (authorization) karena ini adalah pertama kali script dijalankan. Ikuti langkah-langkah authorization: pilih akun Google Anda, klik "Advanced", lalu "Go to SEMS Backend (unsafe)". Script ini akan otomatis membuat 8 sheet di spreadsheet Anda: devices, telemetry\_live, telemetry\_history, automation\_rules, schedules, users, alarms, dan configurations. Setiap sheet akan diisi dengan header kolom dan data default yang meliputi user admin, 17 device default, dan konfigurasi sistem awal.

**PENTING:** Fungsi setupSheets\_() hanya boleh dijalankan SEKALI pada awal setup. Jika dijalankan ulang, data yang sudah dimasukkan (termasuk data telemetry) akan tetap aman karena script akan melewati sheet yang sudah memiliki data.

### 3.6 Deploy sebagai Web App

Setelah semua file disalin dan SPREADSHEET\_ID dikonfigurasi, saatnya mendeploy backend sebagai Web App agar dapat diakses via URL oleh firmware ESP32 dan frontend Next.js. Klik tombol "Deploy" di pojok kanan atas editor Apps Script, lalu pilih "New Deployment". Pada dialog deployment, atur pengaturan berikut dengan teliti karena kesalahan pada langkah ini akan menyebabkan backend tidak dapat diakses oleh perangkat eksternal:

- Klik ikon gear di samping "Select type" dan pilih **Web App**

- Pada **Execute as**, pilih **Me** (akun Anda sendiri)
- Pada **Who has access**, pilih **Anyone** (penting! Agar ESP32 bisa mengakses)
- Klik **Deploy** dan tunggu proses selesai
- Salin **Web App URL** yang muncul - ini adalah GAS\_SCRIPT\_URL Anda

**CATATAN:** Jika Anda memilih "Anyone with Google Account" alih-alih "Anyone", firmware ESP32 tidak akan bisa mengakses API karena tidak memiliki Google Account. Pastikan selalu memilih "Anyone" untuk akses publik.

### 3.7 Catat URL Deployment

URL deployment yang Anda salin di langkah sebelumnya memiliki format sebagai berikut. URL ini sangat penting dan akan digunakan di konfigurasi firmware (platformio.ini) serta environment variable frontend (.env.local). Simpan URL ini di tempat yang aman dan mudah diakses karena Anda akan memerlukannya di beberapa langkah selanjutnya.

```
GAS_SCRIPT_URL:
https://script.google.com/macros/s/AKfycbxE...xY/exec
```

Simpan URL ini! Anda akan memerlukannya untuk:

- build\_flags di platformio.ini (firmware)
- GAS\_SCRIPT\_URL di .env.local (frontend)

## 4. Deploy Frontend - Next.js

Frontend SEMS adalah dashboard web yang dibangun menggunakan Next.js 16 dengan React 19, Tailwind CSS 4, dan komponen UI shadcn/ui. Frontend ini menampilkan data realtime dari backend, menyediakan halaman kontrol relay dan MOSFET, manajemen user, penjadwalan, dan pengaturan konfigurasi sistem. Ada dua opsi deployment yang tersedia: Vercel (cloud, paling mudah) dan VPS dengan Caddy (self-hosted). Pilih salah satu sesuai kebutuhan Anda.

### 4.1 Opsi A: Deploy ke Vercel (Paling Mudah)

Vercel adalah platform cloud yang menyediakan hosting gratis untuk project Next.js. Deployment ke Vercel sangat mudah karena prosesnya hampir otomatis setelah Anda push kode ke GitHub. Pastikan Anda sudah membuat akun Vercel di vercel.com dan menghubungkannya dengan akun GitHub Anda sebelum memulai langkah-langkah berikut.

#### 4.1.1 Install Dependencies

Extract folder frontend\_v7/ dari file zip project ke direktori kerja Anda. Buka terminal atau command prompt, arahkan ke folder frontend\_v7, lalu jalankan perintah berikut untuk menginstall semua dependensi yang diperlukan. Proses ini memerlukan koneksi internet dan memakan waktu beberapa menit tergantung kecepatan koneksi Anda.

```
cd frontend_v7
bun install

# Atau jika menggunakan npm:
npm install
```

#### 4.1.2 Buat File .env.local

Buat file baru bernama .env.local di root folder frontend\_v7/. File ini berisi variabel lingkungan yang dibutuhkan frontend untuk berkomunikasi dengan backend. Isi file tersebut dengan konfigurasi berikut, ganti GAS\_SCRIPT\_URL dengan URL yang Anda catat di langkah 3.7. COOKIE\_SECRET harus berupa string acak minimal 32 karakter untuk keamanan session.

```
GAS_SCRIPT_URL=https://script.google.com/macros/s/AKfycbxE...xY/exec
COOKIE_SECRET=ubah_ini_string_acak_minimal_32_karakter_2024!
```

#### 4.1.3 Build dan Deploy

Jalankan perintah build untuk memastikan tidak ada error. Kemudian push ke repository GitHub baru atau existing, dan hubungkan dengan Vercel. Set environment variables di dashboard Vercel (Settings > Environment Variables) dengan nilai yang sama seperti di .env.local. Vercel akan otomatis mendetect Next.js dan memulai proses build serta deploy.

```
bun run build

# Inisialisasi git dan push ke GitHub:
git init
git add .
git commit -m "Initial SEMS frontend v0.5.0"
git remote add origin https://github.com/username/sems-frontend.git
git push -u origin main

# Atau deploy langsung via CLI (tanpa GitHub):
npx vercel --prod
```

## 4.2 Opsi B: Deploy ke VPS dengan Caddy

Jika Anda lebih suka mengelola server sendiri (self-hosted), Anda bisa menggunakan VPS dengan Caddy sebagai reverse proxy. Caddy menyediakan HTTPS otomatis tanpa konfigurasi sertifikat yang rumit. Pastikan VPS Anda memiliki akses root atau sudo dan telah terinstall sistem operasi Linux (Ubuntu 20.04+ atau Debian 11+ direkomendasikan).

### 4.2.1 Install dan Build di VPS

Salin seluruh folder frontend\_v7/ ke VPS menggunakan scp atau rsync. Kemudian install Bun runtime (atau Node.js) dan jalankan install serta build. Pastikan semua dependensi terinstall dengan benar sebelum melanjutkan ke konfigurasi server web.

```
# Copy file ke VPS:
scp -r frontend_v7/ user@your-vps-ip:/home/user/

# Di VPS, install Bun:
curl -fsSL https://bun.sh/install | bash

# Install dan build:
cd /home/user/frontend_v7
bun install
bun run build
```

#### 4.2.2 Konfigurasi Caddy

File Caddyfile sudah disediakan di folder frontend\_v7/. File ini mengkonfigurasi Caddy untuk melakukan reverse proxy dari port 80/443 ke port 3000 (dimana Next.js berjalan). Caddyfile juga sudah menyertakan security headers (X-Content-Type-Options, X-Frame-Options) dan rate limiting untuk melindungi API dari brute force. Salin Caddyfile ke /etc/caddy/ dan sesuaikan domain dengan domain Anda.

```
# Salin Caddyfile:
sudo cp frontend_v7/Caddyfile /etc/caddy/Caddyfile

# Edit domain:
sudo nano /etc/caddy/Caddyfile

# Isi Caddyfile (sesuaikan domain):
sems.yourdomain.com {
  reverse_proxy localhost:3000 {
    header_up Host {host}
    header_up X-Real-IP {remote_host}
    header_up X-Forwarded-For {remote_host}
    header_up X-Forwarded-Proto {scheme}
  }
}
```

#### 4.2.3 Jalankan Service

Jalankan aplikasi Next.js di background menggunakan process manager seperti pm2 atau systemd. Caddy akan secara otomatis mendapatkan sertifikat SSL dari Let's Encrypt dan melakukan redirect HTTP ke HTTPS. Berikut perintah untuk menjalankan Next.js dan memulai Caddy server. Setelah kedua service berjalan, frontend dapat diakses melalui <https://sems.yourdomain.com>.

```
# Jalankan Next.js (di port 3000):
NODE_ENV=production bun .next/standalone/server.js &

# Atau dengan pm2:
pm2 start "bun run start" --name sems-frontend
```

```
# Install dan mulai Caddy:
caddy install
sudo systemctl start caddy
sudo systemctl enable caddy

# Pastikan Caddy berjalan:
sudo systemctl status caddy
```

## 5. Deploy Firmware - ESP32

Firmware SEMS v7.0.0 adalah program yang berjalan di mikrokontroler ESP32 WROOM. Firmware ini bertanggung jawab untuk membaca semua sensor (INA219, ADS1115, SHT31, DS3231, ACS712, PIR), mengelola relay melalui shift register 74HC595, menjalankan load shedding, automation, dan mengirim data telemetry ke backend setiap 15 detik. Proses deploy firmware memerlukan PlatformIO IDE dan koneksi USB ke ESP32.

### 5.1 Install PlatformIO

Download dan install VS Code dari [code.visualstudio.com](https://code.visualstudio.com) jika belum terinstall. Setelah VS Code terbuka, buka Extensions (Ctrl+Shift+X atau Cmd+Shift+X di Mac), cari "PlatformIO IDE" dan klik Install. PlatformIO akan otomatis menginstall toolchain ESP32 dan library yang diperlukan saat pertama kali Anda membuka project. Proses ini memerlukan koneksi internet dan mungkin memakan waktu 10-20 menit pada kali pertama karena harus mendownload cross-compiler.

**TIPS:** Jika instalasi PlatformIO sangat lambat, pastikan koneksi internet stabil. Anda juga bisa menggunakan PlatformIO CLI sebagai alternatif: "pip install platformio" lalu jalankan "pio run" di folder project tanpa VS Code.

### 5.2 Buka Project Firmware

Extract folder firmware\_v7/ dari file zip project. Buka VS Code, pilih File > Open Folder, lalu navigasi ke folder SEMS\_Firmware di dalam firmware\_v7/. PlatformIO akan otomatis mendeteksi file platformio.ini dan mengkonfigurasi project. Anda akan melihat ikon PlatformIO di sidebar kiri VS Code yang menunjukkan status project, library, dan device yang terhubung.

### 5.3 Konfigurasi Credentials

Buka file platformio.ini dan tambahkan build\_flags di bagian [env:esp32dev]. Credentials ini akan meng-override placeholder CHANGE\_ME di config.h. Pastikan semua nilai diapit oleh tanda kutip

ganda di dalam tanda kutip tunggal. Berikut adalah konfigurasi lengkap yang harus ditambahkan ke platformio.ini Anda:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
upload_speed = 921600
build_flags =
-DDEBUG_ENABLED=1
-DWIFI_SSID=' "nama_wifi_anda" '
-DWIFI_PASSWORD=' "password_wifi_anda" '
-DGAS_SCRIPT_URL=' "https://script.google.com/macros/s/YOUR_ID/exec" '
-DGAS_API_TOKEN=' "sems_device_token_change_me_2024" '
-DOTA_PASSWORD=' "password_ota_anda" '
```

Pastikan nilai GAS\_API\_TOKEN di build\_flags sama dengan device\_token yang ada di sheet configurations di Google Spreadsheet (atau nilai default: sems\_device\_token\_change\_me\_2024 jika belum diubah). Ketidakcocokan token ini adalah penyebab paling umum kegagalan pengiriman telemetry dari firmware ke backend.

## 5.4 Upload ke ESP32

Hubungkan ESP32 ke komputer menggunakan kabel USB. Pastikan kabel USB mendukung transfer data (bukan hanya charging). Jika ESP32 terdeteksi, PlatformIO akan menampilkan nama board di status bar. Klik tombol Upload di PlatformIO (ikon panah ke bawah di bagian bawah VS Code) atau tekan Ctrl+Alt+U. PlatformIO akan mengkompilasi firmware (proses pertama bisa memakan waktu 3-5 menit) dan mengupload ke ESP32. Tunggu hingga muncul pesan "SUCCESS" di konsol output. Jika upload gagal, coba tekan tombol BOOT pada ESP32 saat upload dimulai.

## 5.5 Verifikasi Firmware

Setelah upload berhasil, buka Serial Monitor untuk memverifikasi firmware berjalan dengan benar. Klik ikon monitor di PlatformIO (ikon monitor serial di bagian bawah VS Code) atau gunakan shortcut Ctrl+Alt+S. Pastikan baud rate diatur ke 115200 sesuai konfigurasi di platformio.ini. Anda akan melihat output log firmware yang menampilkan informasi boot, status koneksi WiFi, dan pesan pengiriman telemetry. Berikut adalah tanda-tanda bahwa firmware berhasil berjalan dan terhubung ke backend:

```
Serial Monitor Output (baud rate: 115200):

[SEMS] Boot v7.0.0 | Build: Jun 15 2025 10:30:00
```

```
[SEMS] Connecting to WiFi: nama_wifi_anda  
[SEMS] WiFi connected! RSSI: -45 dBm  
[SEMS] IP: 192.168.1.100  
[SEMS] NTP synced: 2025-06-15 10:30:15 WIB  
[SEMS] Telemetry sent OK (200)  
[SEMS] Config sync OK
```

## 6. Konfigurasi dan Pengujian

Setelah ketiga komponen berhasil di-deploy (backend, frontend, dan firmware), saatnya melakukan konfigurasi awal dan pengujian untuk memastikan seluruh sistem berjalan dengan baik. Bagian ini mencakup pembuatan user admin, verifikasi koneksi firmware, dan pengujian fitur-fitur utama dashboard. Ikuti setiap langkah secara berurutan untuk hasil terbaik.

### 6.1 Buat User Admin Pertama

Buka frontend di browser (alamat Vercel atau domain VPS Anda). Anda akan diarahkan ke halaman login. Gunakan credentials default berikut untuk login pertama kali. User admin default sudah dibuat otomatis oleh fungsi `setupSheets_()` saat inisialisasi backend di langkah 3.5. Password default sengaja dibuat mudah ditebak agar Anda bisa login langsung, namun HARUS segera diubah untuk keamanan sistem.

```
Username: admin  
Password: changeme_immediately
```

**PENTING - KEAMANAN:** Langkah pertama setelah login adalah SEGERA mengubah password default. Buka halaman Settings (ikon gear) di dashboard, cari bagian "Change Password", masukkan password lama (`changeme_immediately`) dan buat password baru yang kuat (minimal 12 karakter, kombinasi huruf besar, kecil, angka, dan simbol).

### 6.2 Konfigurasi Device Token

Buka halaman Settings di dashboard frontend. Di bagian konfigurasi, cari key "`device_token`". Pastikan nilai `device_token` di sheet configurations sama persis dengan `GAS_API_TOKEN` yang Anda set di `build_flags` platformio.ini. Secara default, keduanya bernilai "`sems_device_token_change_me_2024`". Jika Anda mengubah salah satunya, Anda harus mengubah yang lainnya juga agar firmware bisa mengautentikasi request ke backend. Token ini digunakan oleh backend untuk memverifikasi bahwa request berasal dari perangkat ESP32 yang sah.

## 6.3 Verifikasi Koneksi Firmware ke Backend

Buka Serial Monitor di PlatformIO (Ctrl+Alt+S, baud rate 115200). Perhatikan output log firmware. Setelah ESP32 terhubung ke WiFi dan berhasil mengirim data, Anda akan melihat pesan "Telemetry sent OK (200)" yang muncul setiap 15 detik. Pesan ini menandakan bahwa firmware berhasil mengirim data ke backend dan backend merespons dengan sukses. Secara paralel, buka Google Spreadsheet Anda dan periksa sheet "telemetry\_live". Data baru seharusnya muncul di baris terakhir sheet tersebut setiap 15 detik.

- Serial Monitor menampilkan: [SEMS] Telemetry sent OK (200)
- Sheet telemetry\_live di Spreadsheet menunjukkan baris data baru
- Field timestamp, battery\_voltage, battery\_current, soc\_percent terisi
- Jika tidak ada data, periksa GAS\_SCRIPT\_URL dan GAS\_API\_TOKEN di firmware

## 6.4 Test Frontend Dashboard

Setelah firmware berhasil mengirim data, buka dashboard frontend di browser. Dashboard menampilkan ringkasan status sistem termasuk tegangan baterai, arus, SOC (State of Charge), suhu ruangan, dan status relay. Berikut halaman-hal yang perlu Anda periksa untuk memastikan semua fitur berfungsi dengan baik. Jika ada data yang kosong atau error, periksa kembali konfigurasi GAS\_SCRIPT\_URL dan COOKIE\_SECRET di environment frontend.

- **Dashboard (utama)** - Pastikan kartu Battery, Cell Voltages, Inverter, Environment, dan Active Loads menampilkan data
- **Devices** - Daftar 17 perangkat default seharusnya muncul (13 relay + 4 MOSFET)
- **Alarms** - Halaman alarm menampilkan daftar alarm (kosong jika tidak ada masalah)
- **History** - Grafik historis tegangan, arus, dan SOC dari data telemetry
- **Settings** - Konfigurasi sistem, user management, dan pengaturan notifikasi

## 7. Troubleshooting

Bagian ini berisi daftar masalah yang paling sering ditemui selama proses deployment dan operasional SEMS, beserta kemungkinan penyebab dan solusi untuk mengatasinya. Jika Anda mengalami masalah yang tidak tercantum di sini, periksa log Serial Monitor (firmware), log browser Console (frontend), dan execution log di Google Apps Script (backend) untuk mendapatkan informasi lebih detail tentang sumber masalah.



| Masalah                           | Kemungkinan Penyebab                                 | Solusi                                                                                                                          |
|-----------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Firmware tidak connect WiFi       | SSID atau password salah di build_flags              | Periksa build_flags di platformio.ini. Pastikan SSID dan password sesuai, apit dengan tanda kutip ganda di dalam kutip tunggal. |
| Telemetry tidak terkirim          | GAS_SCRIPT_URL salah atau belum di-deploy            | Pastikan URL deployment backend sudah benar. Buka URL di browser - harus menampilkan "SEMS API v1.5.0" atau respons JSON.       |
| Login gagal                       | User belum dibuat di spreadsheet atau password salah | Buka spreadsheet, periksa sheet "users". Jalankan setupSheets_() jika kosong. Reset password jika perlu.                        |
| Dashboard kosong / tidak ada data | Device token tidak cocok antara firmware dan backend | Samakan GAS_API_TOKEN di platformio.ini dengan device_token di sheet configurations.                                            |
| OTA update gagal                  | Password OTA salah atau ESP32 tidak terjangkau       | Periksa OTA_PASSWORD di platformio.ini dan upload_flags. Pastikan ESP32 dan PC di jaringan WiFi yang sama.                      |
| Data tidak muncul di sheet        | Backend belum di-deploy sebagai Web App              | Deploy ulang di Apps Script: Deploy > New Deployment > Web App. Pastikan "Anyone" untuk akses.                                  |
| Frontend error 500                | COOKIE_SECRET belum di-set di environment            | Tambah COOKIE_SECRET di .env.local atau environment variables Vercel. String acak min 32 karakter.                              |
| Sensor tidak terbaca              | Kabel I2C (SDA/SCL) longgar atau salah pin           | Periksa koneksi SDA (GPIO 21) dan SCL (GPIO 22). Pastikan pull-up resistor terpasang.                                           |

## 8. Keamanan - Checklist Wajib

Keamanan adalah aspek kritis dalam sistem IoT seperti SEMS. Sistem ini mengelola perangkat listrik rumah tangga melalui relay 220V, sehingga akses tidak sah dapat berakibat fatal. Berikut adalah checklist keamanan yang WAJIB Anda selesaikan setelah deployment. Tandai setiap item yang sudah selesai dan pastikan semua item terpenuhi sebelum sistem digunakan secara produksi. Jangan abaikan item manapun karena masing-masing menutup celah keamanan yang berbeda.

| Item Keamanan                                        | Detail dan Cara Melakukan                                                                                                                                                          |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div> <div></div> Ubah default admin password </div> | Password default "changeme_immediately" harus SEGERA diubah melalui halaman Settings. Gunakan password minimal 12 karakter dengan kombinasi huruf besar, kecil, angka, dan simbol. |

|                                                                                      |                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>■ Set COOKIE_SECRET yang kuat</li> </ul>      | <p>Buat string acak minimal 32 karakter. Gunakan perintah "openssl rand -base64 32" di terminal untuk generate string acak yang aman. Set di .env.local dan environment Vercel.</p> |
| <ul style="list-style-type: none"> <li>■ Ubah OTA_PASSWORD dari default</li> </ul>   | <p>Password OTA default "CHANGE_ME" memungkinkan siapapun di jaringan WiFi mengupload firmware baru ke ESP32 Anda. Set password yang kuat di build_flags platformio.ini.</p>        |
| <ul style="list-style-type: none"> <li>■ Ubah device_token dari default</li> </ul>   | <p>Token default "sems_device_token_change_me_2024" harus diganti dengan string unik. Ubah di sheet configurations (backend) dan build_flags (firmware) secara bersamaan.</p>       |
| <ul style="list-style-type: none"> <li>■ Pastikan HTTPS aktif di frontend</li> </ul> | <p>Frontend HARUS diakses via HTTPS, bukan HTTP. Vercel otomatis menggunakan HTTPS. Untuk VPS, Caddy otomatis mendapatkan sertifikat SSL dari Let's Encrypt.</p>                    |
| <ul style="list-style-type: none"> <li>■ Batasi akses user</li> </ul>                | <p>Jangan membuat user dengan role "viewer" atau "technician" secara berlebihan. Setiap user yang terdaftar memiliki akses ke sistem. Hapus user yang tidak aktif.</p>              |